

Refinamiento Adaptativo de Octree en CUDA

Representación Volumétrica Paralela de Mallas 3D

Juan García

Universidad Técnica Federico Santa María

Profesor: Jorge Díaz

25 de junio de 2026

Contenido

- 1 Motivación
- 2 Problema Computacional
- 3 Arquitectura del Proyecto
- 4 Clasificación Geométrica
- 5 Paralelización
- 6 Metodología
- 7 Resultados
- 8 Comparativa CPU vs. GPU
- 9 Análisis Comparativo
- 10 Limitaciones y Trabajo Futuro
- 11 Conclusión

¿Por qué usar un Octree?

El Problema de la Grilla Uniforme

Una grilla uniforme refina zonas **vacías** con el mismo detalle que zonas **cercanas a la superficie**.

Grilla Uniforme:

- ✗ Crece exponencialmente en memoria.
- ✗ Procesa regiones vacías innecesariamente.
- ✗ No escala para alta resolución.

Octree Adaptativo:

- ✓ Concentra resolución en la superficie.
- ✓ Elimina cubos vacíos mediante poda.
- ✓ CUDA paraleliza el refinamiento y la clasificación.

La salida **VTK** facilita inspeccionar el resultado en herramientas como ParaView.

Definición del Problema

Entrada

Una malla triangular en formato .mdl (modelo cortex.mdl).

Proceso por cada nivel de refinamiento:

- 1 Cada cubo sobreviviente genera **8 hijos**.
- 2 Cada hijo se clasifica respecto de la malla:

FUERA

No pertenece al volumen. **Se elimina.**

DENTRO

Completamente interior. **Sobrevive.**

BORDE

Intersecta la superficie. **Sobrevive.**

Desafío: Crecimiento Exponencial

Sin poda, la cantidad de cubos crece como 8^{nivel}

Los cubos FUERA deben ser eliminados para mantener la viabilidad computacional.

Nivel	Cubos sin poda
3	512
6	262.144
9	134.217.728

El objetivo es evitar procesar y almacenar regiones vacías,
manteniendo una representación detallada de la geometría.

Árbol del proyecto

```
paralelo/  
  Makefile  
  cortex.mdl  
  src/  
    main.cu  
    octree.cu / octree.cuh  
    mesh_classify.cu/.cuh  
    geometry.cu / geometry.cuh  
    mesh.h  
    vtk_writer.cpp/.h  
  output/  
    octree_pruned_level_*.vtk
```

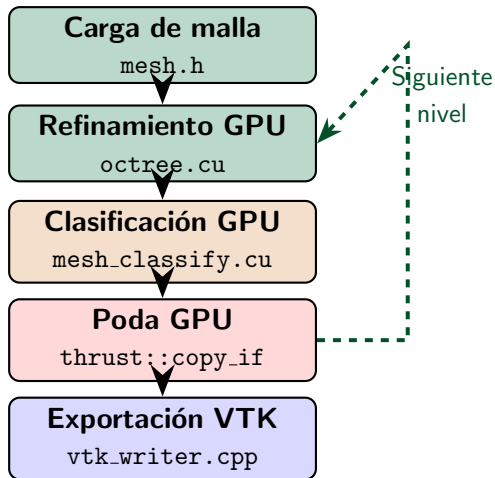
Flujo principal (main.cu):

- 1 Cargar malla .mdl.
- 2 Copiar vértices y triángulos a GPU.
- 3 Construir cubo raíz desde bounding box.
- 4 Por nivel: Refinar → Clasificar → Podar.
- 5 Copiar resultado final a CPU.
- 6 Exportar archivo .vtk.

Componentes del Sistema

Componente	Responsabilidad Principal
<code>mesh.h</code>	Carga malla <code>.mdl</code> (vértices/triángulos) y calcula bounding box.
<code>octree.cu</code>	Refinamiento de cubos y poda paralela con <code>thrust::copy_if</code> .
<code>mesh_classify.cu</code>	Clasificación paralela (AABB vs Malla) usando SAT y Ray Casting.
<code>geometry.cu</code>	Pruebas secundarias (e.g., clasificación contra esfera).
<code>vtk_writer.cpp</code>	Exportación a hexaedros VTK para visualización en ParaView.
<code>Makefile</code>	Compilación optimizada con <code>nvcc -O2 -arch=sm_89</code> .

Flujo de la Arquitectura



Clasificación: Dos Etapas

Cada hilo CUDA procesa un cubo de forma independiente.

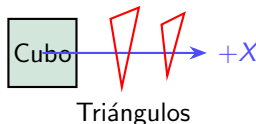
Etapas 1: Triángulo vs. AABB

- Se aplica SAT (Separating Axis Theorem).
- Basado en el test de Akenine-Möller.
- Si algún triángulo intersecta el cubo:
⇒ Cubo marcado como **BORDE**.

Etapas 2: Ray Casting

- Solo si no hay intersección.
- Rayo en dirección $+X$ desde el centro del cubo.
- Número **impar** de cruces: **DENTRO**.
- Número **par** de cruces: **FUERA**.

2 cruces ⇒ **FUERA**



Estrategia de Paralelismo

Paralelismo de Datos

Cada cubo es una unidad de trabajo independiente, lo que permite un mapeo directo a hilos CUDA.

Refinamiento

1 hilo = 1 cubo padre
Genera sus 8 hijos.

Clasificación

1 hilo = 1 cubo hijo
Evalúa SAT + ray casting.

Poda

`thrust::copy_if`
Compacta cubos vivos en GPU.

Acceso a Memoria

La estructura Cube se almacena en **arreglos planos**, evitando punteros de árbol y favoreciendo acceso secuencial en memoria de GPU.

Ejecución del programa:

```
make  
make run LEVEL=N MDL=cortex.mdl
```

Para cada nivel se realiza el ciclo completo:

- ➊ Refinar → Clasificar → Podar.
- ➋ Registrar la cantidad final de cubos sobrevivientes.
- ➌ Exportar output/octree_pruned_level_N.vtk.
- ➍ Comparar contra el caso sin poda (8^{nivel}).

Métrica de Ahorro

$$\text{ahorro} = 100 \times \left(1 - \frac{\text{cubos sobrevivientes}}{8^{\text{nivel}}} \right)$$

Resultados: Cubos y Ahorro

Nivel	Con poda	Sin poda	Ahorro	Archivo VTK
3	259	512	49,4 %	77 KB
4	1.572	4.096	61,6 %	470 KB
5	10.646	32.768	67,5 %	3,2 MB
6	78.019	262.144	70,2 %	24 MB
7	596.068	2.097.152	71,6 %	186 MB
8	4.653.818	16.777.216	72,3 %	567 MB
9	36.775.635	134.217.728	72,6 %	12 GB

Tendencia del Ahorro

La poda elimina aproximadamente entre **49 %** y **73 %** de los cubos según el nivel.

Observaciones positivas:

- El ahorro *aumenta* con la resolución porque gran parte del dominio corresponde a espacio vacío.
- La poda permite manejar niveles que serían imposibles con grilla uniforme.

Observaciones críticas:

- Aun con poda, el crecimiento sigue siendo alto: nivel 9 produce más de 36 millones de cubos.
- La salida ASCII VTK llega a **12 GB** en nivel 9.

Resultados: Tiempos de Ejecución en GPU

Nivel	Cubos Antes	Cubos Clasif.	Cubos Fuera	Cubos Sobrev.	Tiempo GPU (4050)
1	8	8	0	8	168.04 ms
2	64	64	0	64	1.08 ms
3	259	512	253	259	5.50 ms
4	1.572	2.072	500	1.572	16.52 ms
5	10.646	12.576	1.930	10.646	87.89 ms
6	78.019	85.168	7.149	78.019	582.49 ms
7	596.068	624.152	28.084	596.068	4.221,75 ms (4,22 s)

Nota: El tiempo de Nivel 1 incluye la sobrecarga inicial de compilación JIT y reserva de memoria en CUDA.

Comparativa de Rendimiento: CPU vs. GPU

Características del Hardware en Uso

- **CPU:** Intel Core i7-13700H (14 núcleos / 20 hilos virtuales @ 3.5 GHz)
- **GPU:** NVIDIA GeForce RTX 4050 Laptop (2560 núcleos CUDA @ 2.0 GHz)

Nivel	Cubos	CPU Sec.	CPU 20 Hilos	GPU (4050)	Speedup
Nivel 6	85K	20,81 s	$\approx 1,38 \text{ s}^*$	0,58 s	35,7x
Nivel 7	624K	166,07 s	$\approx 11,00 \text{ s}^*$	4,22 s	39,3x
Nivel 8	4,77M	$\approx 21,1 \text{ min}^*$	$\approx 1,40 \text{ min}^*$	31,6 s	40,1x*
Nivel 9	37,23M	$\approx 2,75 \text{ h}^*$	$\approx 10,9 \text{ min}^*$	246,5 s	40,2x*

Nota: Los tiempos secuenciales de CPU para Niveles 6 y 7 son reales (medidos). Los tiempos de CPU Sec. para Niveles 8 y 9 son proyecciones basadas en la cantidad de cubos. Los tiempos multi-hilo (CPU 20 Hilos) son estimaciones teóricas (escala $\approx 15\times$). El speedup es respecto al caso secuencial.

Cumplimiento de la Propuesta

Objetivo	Estado
Construir octree adaptativo	✓ Cumplido
Ejecutar refinamiento en CUDA	✓ Cumplido
Clasificar cubos contra geometría 3D	✓ Cumplido
Podar cubos fuera de la malla	✓ Cumplido
Exportar resultados para visualización	✓ Cumplido
Medir reducción frente a grilla completa	✓ Cumplido

Otras Alternativas Consideradas

Alternativa	Ventajas	Desventajas
Grilla Uniforme 3D	Fácil de implementar y acceder.	Memoria exponencial ($O(N^3)$), procesa vacío.
Octree en CPU	Depuración y diseño sencillos.	Desempeño secuencial, no escala a millones.
Sparse Voxel Octree (SVO)	Alta eficiencia y compactación.	Complejidad extrema de implementación en GPU.

Justificación de la elección

El **Octree Adaptativo en CUDA** equilibra una estructura simple (arreglos planos) con alto paralelismo en GPU, reduciendo la memoria en zonas vacías y exportando volúmenes compatibles con ParaView.

Limitaciones Observadas

Limitaciones Actuales

- Clasificación costosa al revisar todos los triángulos por cada cubo.
- Salida VTK **ASCII** muy pesada (hasta 12 GB en nivel 9).
- Falta instrumentación experimental de tiempos de ejecución.
- Sensibilidad a mallas abiertas (puede alterar el ray casting).

Trabajo Futuro

- Implementar formato VTK binario comprimido.
- Usar estructuras de aceleración (BVH o grillas uniformes)
- Refinamiento adaptativo selectivo en celdas de borde.

Ventajas de GPU frente a CPU

Paralelismo de Datos Masivo

Cada celda del octree es independiente. La GPU ejecuta miles de pruebas geométricas (SAT y Ray Casting) de forma concurrente, evitando el cuello de botella secuencial de la CPU.

Eficiencia en el Ancho de Banda

La malla 3D se copia una sola vez a la memoria global de la GPU, permitiendo que miles de hilos accedan en paralelo con muy baja latencia de caché L2/Textura.

Aceleración Exponencial (Speedup)

Permite procesar niveles de resolución fina en segundos (e.g., Nivel 9 con 37M de cubos en 246,5 s vs. las $\approx 2,75$ horas estimadas en CPU).

Pipeline Funcional

El proyecto implementa un pipeline completo de octree adaptativo en CUDA para discretizar una malla 3D.

Puntos Clave:

- La poda reduce entre **49 % y 73 %** de los cubos frente a una grilla completa.
- CUDA permite escalar el refinamiento a decenas de millones de cubos.
- La arquitectura cumple la propuesta y deja base clara para extensiones.
- El tamaño de salida VTK escala agresivamente: se necesita formato binario.

La estrategia de poda concentra la resolución donde importa, haciendo viable la representación volumétrica de alta resolución.

Fin

Gracias por su atención

Proyecto Paralelo
Universidad Técnica Federico Santa María